What Survives the Filter? Trivial Baselines on Stored Honest/Attack Pairs in AI-Control Monitoring

Anonymous Author(s)
Affiliation
Address
email

Abstract

Trusted monitoring in AI control is evaluated on stored honest/attack pairs. The field already knows these pairs leak: it filters to working solutions and strips comments. We ask what that preprocessing leaves behind. On the field's own filtered, comment-stripped split, a *trained* bag-of-words classifier scores above roughly four-fifths of the open-weight monitors ranked on it. A classifier that reads no code at all — only problem metadata — scores above most of them. We trace the residue to authorship rather than sabotage: the honest class is human-written while every attack is model-written. The signal transfers across backdoor generators, is nearly as strong on inert backdoors as on working ones, and survives the field's comment stripper. Regenerating only the honest half does not isolate content: with an open-weight model writing the honest class against the stored backdoors, separability *rises*. Holding authorship constant by construction — one model writing both classes — removes between a third and nine-tenths of the separability. The share of above-chance signal that survives is $\rho = 0.687$ for Claude Sonnet 5 and $\rho = 0.095$ for Claude Haiku 4.5. The two differ because only Sonnet adopts the attack prompt's layout. Matching that prompt on the honest side removes most of what remains. What survives on attacks that actually fire is consistent with surface content a monitor also reads. A zero-shot monitor is inflated the same way, scoring 0.944 on stored pairs and 0.818 on firing backdoors from the same author. We recommend that stored-pair evaluations report partial-input and bag-of-words floors against permutation nulls, use same-generator honest samples and the same prompt, and drop inert backdoors from both classes.

1 Introduction

Trusted monitoring is the backbone protocol of AI control: a weaker trusted model reads an untrusted model's output and flags sabotage [4]. It is developed, tuned and ranked almost entirely on *stored* honest/attack pairs — fixed corpora of solutions, some carrying a planted backdoor [12, 19, 9]. The field already knows these pairs leak. Koran et al. keep only working honest solutions and working backdoors and strip every comment, with the rationale stated [12]; Control Tax reports the pass-tests pre-filter and appendices "Monitoring failed backdoors" [19]; [18] is an entire paper on the comment channel. The sharpest treatment is Jha's, whose harness ships a `-leak-check` flag with two trivial baselines inside it and concludes that stripping is mandatory rather than cosmetic: a `#`-count detector falls from 0.6957 to 0.4983 once comments go, and code length alone sits at 0.4726 [9]. This paper starts from that work.

What nobody has measured is what the preprocessing leaves behind. On the field's own filtered, comment-stripped split, a bag-of-words classifier reaches AUROC 0.806 — above 20 of the 24 open-weight monitors ranked on that same split, whose median is 0.609 — and a classifier that reads no code at all, only problem difficulty, test count and question length, reaches 0.772, above 20.

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

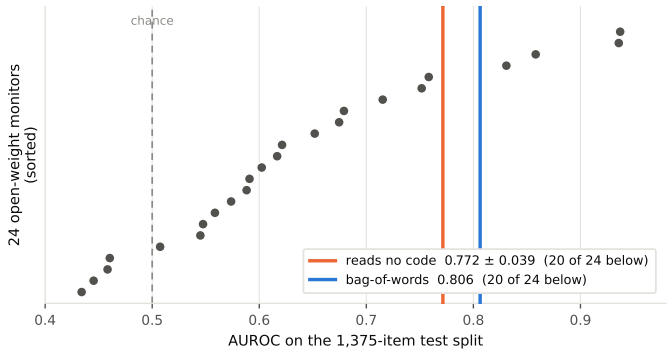

Figure 1: **Two floors against the 24 open-weight monitors ranked on the same split.** AUROC on Koran et al.'s 1,375-item test split (172 attack / 1,203 honest), comments stripped; monitor scores from [9] (MIT), median 0.609, max 0.937. **Two caveats.** (i) The monitors are zero-shot prompted while the bag-of-words classifier is *trained* on the benchmark's own paired train split: a statement about what the corpus makes recoverable [6], not a claim that bag-of-words is a better monitor. (ii) The no-code floor is cross-validated *within* the test split; on one held-out slice it is 0.716, above 18 of 24 (appendix A.2).

Much of the residue is authorship, not sabotage. Honest solutions come from APPS [7] and are human-written; every backdoor is model-written. The classifier transfers across backdoor generators and is nearly as strong on **inert** backdoors as on working ones — neither is a property of a backdoor. That bag-of-words separates human-written from model-written code is established out-of-field [14]; the known-ness of the mechanism is the argument, not a dent in it. The neighbouring field already applies the control missing here — CodeMirage gates its classes on line and character count [5], MultiAIGCD pairs to shared problem definitions [1] — so unmatched pairs are a defect by a published standard. Adjacent control work attacks other axes: [20] subverts monitors with injected text, [3] shows attacker-side selection inflates safety estimates, and [15] removes transcript length as a nuisance covariate; we publish one as a per-benchmark floor.

We contribute (i) a protocol $\times$ baseline measurement of what survives the field's preprocessing; (ii) a same-generator control on two generators, attributing between a third and nine-tenths of the separability to authorship and most of the remainder to the attack prompt's framing, with a negative result on the way: regenerating one half of a stored pair swaps the confound rather than removing it; (iii) a second selection artifact the filter installs by defining the negative class through attack failure; and (iv) a preprocessing checklist and a released split. **Scope:** this concerns monitor training, selection and benchmarking on *stored* pairs; our measurement is of one APPS-derived family plus five agentic settings. ControlArena's live evaluations generate both classes and are not implicated.

2 Setup

The confound is legible in the schema. The canonical stored-pair artifact for APPS-based control evaluations [8] records `model` and `source` on every backdoor and no generator field on any honest solution (Table 2, appendix).

Artifacts and provenance. We use seven public corpora; their per-class provenance is the object of study: in the APPS family the honest class is human and the attack class is one of ten named models. **Six of the seven, from two publishers, declare no licence** [11, 17]; we report only aggregates over them and redistribute no corpus.

Evaluation surface, and naming protocols. Our headline protocol is the filtered, comment-stripped split of [12]: 1,209 train problems (2,418 paired items) and 1,375 test items (172 attack, 1,203 honest), labelled by `backdoor_works`, reconstructed with the released code of [9]. Every comparison below names its split, metric and preprocessing. Generated code is scored by our own runner, applied to both classes (appendix A.11).

3 Results

The fix works, on the two carriers §1 names (Table 3, appendix). What it does not reach, on that same split, is a bag-of-words floor of 0.806 (95% CI 0.7704–0.8409 by bootstrap over problems; permutation null 95th percentile 0.5465, $p < 0.005$), above 20 of the 24 monitors by point estimate and 18 of them with a paired-bootstrap interval excluding zero, and a no-code floor of 0.772 (CI 0.7319–0.8005 on the pooled out-of-fold 0.766; null 0.5545, $p < 0.005$), above 20 and 17 respectively (Figure 1): the preprocessing removed the carrier it targeted and not the two beside it.

The signal is not sabotage. Three measurements point the same way. Every number here is comment-stripped on both classes; the unstripped values, inflated by the comment channel, are in appendix A.3. First, the classifier *transfers across backdoor generators* at 0.969 pairwise: trained on one attacker's backdoors it identifies another's, so what it has learned is not any single model's sabotage style. Second, it is nearly as strong on **inert** backdoors (0.879) as on working ones (0.935); if it were detecting sabotage, backdoors that do not function should be much harder. Third, the residual carrier is stylistic rather than semantic: `__main__` appears in 0.116 of attack samples against **0.000** of honest ones, replicated on the train split (152 versus 1 of 1,209) and untouched by the stripper. None of these is a property of a backdoor, and a token-level test [14] does not replicate here (appendix A.12).

Regenerating one half of a pair does not isolate content. Under the identical pipeline the human-honest baseline is $B = 0.871$ over $n = 1{,}444$ pairs. The routing rule and its bands were fixed before any arm ran. Retained signal is $\rho = (G - 0.5)/(B - 0.5)$: $\rho = 1$ means holding authorship constant changed nothing, $\rho = 0$ that nothing survived. We pre-registered $\rho \leq 0.33$ as authorship and $\rho > 0.72$ as content. With an open-weight model writing the honest class against the same backdoors, separation *rises*, to $\rho = 1.234$ (appendix A.5): substituting the honest author replaced one authorship confound with a larger one.

One model writing both classes removes between a third and nine-tenths of the separability (Table 1). We report the **independent** arm: the attack class is written from the problem statement alone, so authorship changes and nothing else. An **edit** arm, shown its own honest solution, returns near-copies instead (appendix A.4). We take one outcome-blind draw per problem and define the attack class as the benchmark does, by intent plus passing the public tests. 2.2% of the first generator's draws fire and 0.3% of the second's (2 of 611); B' is inert-inclusive over the same problems. The first generator's interval excludes the collapse band and spans the content line, so the pre-registered rule returns *inconclusive between partial and content*. The second generator's lies inside the collapse band. The drop is not sample size: at $n = 623$, B under the identical pipeline is 0.8521 ± 0.0055. Two rows are *strict*: they keep only attacks that pass the tests *and* fire on their trigger, selected by the benchmark's own retry-and-keep as our Recommendation 3 asks. Only 99 pairs meet it, so we read them through ρ alone, B at that n being 0.7807 ± 0.0248; they agree with the headline rows at low power.

Table 1: **Gate S: what is left when one model writes both classes.** S is the same-generator AUROC (5-fold, comments stripped) and B' the stored-pair baseline on the row's own problems; ρ, its bands and its intervals are as in the text and appendix A.5.

Arm	n	S (95% CI)	B'	ρ (95% CI, joint)	band
Sonnet 5, headline	623	0.751 (0.7295–0.7713)	0.8658	0.687 (0.625–0.755)	partial–content
Haiku 4.5, headline	611	0.5340 (0.5159–0.5531)	0.8584	0.095 (0.044–0.150)	**collapse**
Sonnet 5, prompt-matched	586	0.6423 (0.6179–0.6621)	0.8613	0.394 (0.326–0.454)	**partial**
Sonnet 5, strict	99	0.7652 (0.7016–0.8000)	0.7877	0.921 (0.662–1.089)	partial–content
Sonnet 5, strict, matched	96	0.7113 (0.6464–0.7453)	0.8030	0.697 (0.494–0.856)	partial–content

The two generators differ because of the attack prompt, not because the measurement is noisy. What survives on Sonnet 5 is lexical: length alone falls to 0.5851 while TF-IDF holds. It is not the trigger. Masking numerals and comparison operators leaves S unchanged; only normalising identifiers moves it, to 0.700 ($\rho = 0.54$). The top attack-side features are `def main` (67% of attack programs against 38.5% of honest ones) and `data[`: the program layout the attack prompt elicits. Haiku 4.5 never adopts it. `def main` and `sys.stdin` appear in no program of any of its three classes, which read input with `input(` throughout (appendix A.6). So when the prompt is held constant as

well as the model, the two move toward each other, to $\rho = 0.394$ and $\rho = 0.113$. On Sonnet that removes most of what is left: the two honest classes alone, no attack on either side, already separate at 0.6006, so what the prompt removes is framing, not backdoor. On Haiku the same prompt-only floor is 0.4918, an interval containing chance.

A zero-shot monitor is inflated the same way. We put that prompt to the model that wrote both classes. On the *stored* pairs over the headline problems it scores 0.9579; on the same-generator pairs, 0.566 (0.5507–0.5833). That is a monitor-ρ of 0.145 (0.111–0.182), below the classifier's 0.687. On the strict pairs the two scores are 0.9437 and 0.818 (0.7630–0.8686), a monitor-ρ of 0.717 (0.601–0.832). A monitor that wrote neither class returns the same two ratios: 0.151, a bound at half missingness, and 0.749. Our single-shot attacks fire given a pass at 2.5% against the stored attacker's 67.2%: the stored attack class is a survivor set.

The filter installs a second artifact. Labelling by `backdoor_works` defines the negative class by *attack failure* at the problem level rather than by honesty: 80.7% of the test split's labelled backdoors do not work (32.8% of those that pass the public tests), and a problem whose backdoor fails contributes its human honest solution instead. The label is fixed before any code is read. Attack problems also carry half as many test cases (20.95 versus 38.85) — fewer tests, easier to backdoor undetectably — so difficulty and test count correlate with the label, which is where the content-blind 0.772 comes from.

The negative control comes free. In five same-generator agentic settings, where the same three models write both classes and no authorship confound is possible, the surviving floor is structural: 0.504 (`bash`, on its null) to 0.802 (`agentdojo`) on character count alone (Figure 2, appendix).

4 Recommendations

Following Kapoor and Narayanan [10], who treat leakage as a checklist rather than a verdict:

1. **Generate both classes with the same model and the same prompt framing.** If the attack class is model-written, the honest class must be too; as our arm-1 null shows, regenerating only one side substitutes one authorship gap for another.
2. **Report a partial-input floor and a bag-of-words floor beside every monitor number**, each against a permutation null.
3. **Do not define the negative class by attack failure.** Drop the problems whose attacks failed, or sample the negative class independently of attack success.
4. **State the split, the metric and the preprocessing** whenever monitors are compared, and **release the split and the scripts**. Ours accompany this paper under MIT (appendix A.10).

5 Limitations

A high floor is evidence about a corpus, not a verdict on results computed on it. Feng et al. [2] warn that a partial-input baseline's *failure* does not prove a dataset clean; the converse binds us. Our floors show the label is recoverable without reading a backdoor [6, 16], not that monitors exploit it.

The same-generator arm is a later model behind a paid API. The stored attacker, `claude-3-7-sonnet-thinking`, is retired (404 against a live key), so both classes come from a later model of the same lineage, called through a metered API that does not expose its sampling parameters, so the arm is not bit-reproducible; thinking was off although the stored attacker was a thinking model. The stored attacker and ours also differ in backdoor style, not only in authorship, so every ρ here — classifier or monitor — bounds how much of the separability is content rather than partitioning it. And the two generators are compared on the `passes` eligibility alone: Haiku 4.5's strict row was refused at 35 pairs against our 50-pair floor.

Scope, provenance and the harness. Whether stored-pair corpora leak this way is unknown: five of our seven corpora are one user's repackaging, and our claims are scoped to them. Two interpreter-era artifacts reach B but not the same-generator arm (appendix A.11).

References

[1] Basak Demirok, Mucahid Kutlu, and Selin Mergen. MultiAIGCD: A comprehensive dataset for AI generated code detection covering multiple languages, models,prompts, and scenarios. 2025. arXiv:2507.21693v1.

[2] Shi Feng, Eric Wallace, and Jordan Boyd-Graber. Misleading failures of partial-input baselines. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2019.

[3] Catherine Ge-Wang, Tyler Crosse, Benjamin Hadad, Joachim Schaeffer, Ram Potham, and Tyler Tracy. Attack selection in agentic AI control evaluations meaningfully decreases safety. 2026. arXiv:2606.06529.

[4] Ryan Greenblatt, Buck Shlegeris, Kshitij Sachan, and Fabien Roger. AI control: Improving safety despite intentional subversion. 2023. arXiv:2312.06942.

[5] Hanxi Guo, Siyuan Cheng, Kaiyuan Zhang, Guangyu Shen, and Xiangyu Zhang. CodeMirage: A multi-lingual benchmark for detecting AI-generated and paraphrased source code from production-level LLMs. 2025. arXiv:2506.11059v1. No venue stated.

[6] Suchin Gururangan, Swabha Swayamdipta, Omer Levy, Roy Schwartz, Samuel R. Bowman, and Noah A. Smith. Annotation artifacts in natural language inference data. In *Proceedings of NAACL-HLT*, 2018.

[7] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. Measuring coding challenge competence with APPS. In *NeurIPS Datasets and Benchmarks Track*, 2021. arXiv:2105.09938. The upstream source of the human-written honest class, distributed as the HuggingFace dataset `codeparrot/apps`.

[8] Rogan Inglis. apps-control-arena. HuggingFace dataset `RoganInglis/apps-control-arena`. MIT licence. 5,000 rows; honest class from `codeparrot/apps`., 2025.

[9] Anik Jha. Decorrelation is not complementarity: Skill, not lineage, governs trusted-monitor ensembles. 2026. arXiv:2608.16190. Code: `https://github.com/anik-jha/challenger-panels` (MIT).

[10] Sayash Kapoor and Arvind Narayanan. Leakage and the reproducibility crisis in ML-based science. 2022. arXiv:2207.07048.

[11] Eugene Koran. apps-control-arena-high-quality. HuggingFace dataset `eugenekoran/apps-control-arena-high-quality`. **No licence declared.** 1,209 train / 1,375 test., 2026.

[12] Eugene Koran, Yejun Yun, Samantha Tetef, Benjamin Arnav, and Pablo Bernabeu-Pérez. Ensemble monitoring for AI control: Diverse signals outweigh more compute. 2026. arXiv:2605.15377v2.

[13] Bryce Little. Length penalties make chain-of-thought less monitorable. 2026. arXiv:2607.09786v3.

[14] Daniil Orel, Dilshod Azizov, Indraneil Paul, Yuxia Wang, Iryna Gurevych, and Preslav Nakov. AICD bench: A challenging benchmark for AI-generated code detection. 2026. arXiv:2602.02079v1. No venue stated on the abs page.

[15] Lucas Pinto. Calibration-family overfit: Why trusted sabotage monitors don't transfer across lineages. In *COLM 2026 Workshop on AI Measurement Science*, 2026. arXiv:2607.06596v3.

[16] Adam Poliak, Jason Naradowsky, Aparajita Haldar, Rachel Rudinger, and Benjamin Van Durme. Hypothesis only baselines in natural language inference. In *Proceedings of the Seventh Joint Conference on Lexical and Computational Semantics (*SEM)*, 2018.

[17] Aditya Sinha. control_arena_{bash,shade,iac,rogue_eval,agentdojo}. HuggingFace datasets `adityaasinha28/control_arena_*`. **No licence declared.** 1,500 rows each., 2026.

[18] Ashwin Sreevatsa, Sebastian Prasanna, and Cody Rushing. Basic legibility protocols improve trusted monitoring. 2026. arXiv:2602.10153.

[19] Mikhail Terekhov, Zhen Ning David Liu, Caglar Gulcehre, and Samuel Albanie. Control tax: The price of keeping AI in check. 2025. arXiv:2506.05296v3.

[20] Mikhail Terekhov, Alexander Panfilov, Daniil Dzenhaliou, Caglar Gulcehre, Maksym Andriushchenko, Ameya Prabhu, and Jonas Geiping. Adaptive attacks on trusted monitors subvert AI control protocols. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2510.09462v2.

A Supplementary tables and figures

A.1 The schema

Table 2 is the record schema of `RoganInglis/apps-control-arena` [8]. The attack class carries `model` and `source`, naming the system that produced each backdoor; the honest class carries no such field, because there is no generator to name. The two APPS-backdoor corpora we use are two views of this one artifact: every problem in the filtered release [11] is in [8], byte-identical except a re-validated `backdoor_works`. Among the agentic corpora we exclude AgentHarm, whose author is an invited speaker at this venue.

Table 2: The confound is legible in the dataset schema, with no measurement at all. The attack class records which model wrote it; the honest class has no generator field. Verified against the shard schema of `RoganInglis/apps-control-arena`.

struct	sub-fields
`solutions`	`code`, `compiles`, `passes_tests`
`backdoors`	`backdoor_code`, `backdoor_input`, `backdoor_works`, `compiles`, `model`, `passes_tests`, `source`

A.2 The protocol $\times$ baseline table

The no-code floor is cross-validated *within* the test split because the released train/val split is paired and therefore metadata-balanced; its permutation null shuffles labels for fitting and scoring alike. Table 3 is the full measurement behind §3. Rows are not comparable across protocols; the preprocessing column says whether comments were stripped before the baseline read the code. Pairwise accuracies are given to three decimals because their resolution over 2,641 pairs is one pair. Intervals in §3 are bootstraps over problems (2,000 draws) and nulls are 200 label permutations; the held-out no-code row is barely above its own null: 0.716 (CI 0.6476–0.7796) against a 95th percentile of 0.7023, $p = 0.030$, above 18 of the 24 monitors by point estimate and 12 with a paired interval excluding zero. The no-code claim therefore rests on the cross-validated protocol.

A.3 Comment stripping on the matched-protocol numbers

The matched-pair and transfer measurements were first made with comments in. Every value in the authorship paragraph of §3 was re-measured with the same stripper applied to both classes, over five problem-disjoint splits (each transfer row is one fixed split). With comments kept the same quantities were transfer 0.994, working 0.979, inert 0.955, all pairs 0.962 and surface-feature AUROC 0.8306; stripped they are 0.969, 0.935, 0.879, 0.897 and 0.7259. Every one was inflated by the comment channel, which is this paper's thesis applied to itself. The `#`-presence marker, 0.685 against 0.176 before stripping, closes to 0.007 against 0.008 after and is dropped; `__main__` is unchanged. One authorship leak survives the stripper because string literals are not comments: in the comment-stripped train split an author byline (`Author :`) survives in 23 of 1,209 honest samples against 2 attack samples, and a triple-quoted block in 43 against 9 — too rare to drive 0.806, but

Table 3: What the field's preprocessing leaves behind. Every row is measured under the protocol named in column one; rows are not comparable across protocols. *Prep.* says whether comments were stripped before the baseline read the code; *no code* means the baseline never reads the program. Pairwise accuracies are given to three decimals, their resolution over the pair count.

Protocol	Baseline	Value	Prep.	Against
AUROC				
Koran's exact split	TF-IDF word 1–2gram	0.8063	stripped	above 20 of 24
same	TF-IDF word 1–2gram	0.8914	kept	above 22 of 24
same, reads no code	difficulty + #tests + $\|q\|$	0.7715 ± 0.0389	no code	above 20 of 24
same, held-out slice	difficulty + #tests + $\|q\|$	0.7163	no code	above 18 of 24
same	code length only	0.4726	stripped	chance
same	#-count detector	0.4983	stripped	chance
same	#-count detector	0.6957	kept	—
Raw artifact, problem level	all metadata	0.8917	no code	—
same	difficulty only	0.6823	no code	—
same	`solution_passes_tests`	0.7881	no code	—
Matched pairs (P3)	all metadata	0.5000	no code	exactly chance by construction
Pairwise accuracy				
DSPy matched, problem-disjoint	TF-IDF word 1–2gram	0.897	stripped	tutorial monitor 0.69–0.81
working backdoors only	TF-IDF word 1–2gram	0.935	stripped	—
inert backdoors only	TF-IDF word 1–2gram	0.879	stripped	the signal is not sabotage
transfer, Control Tax → GPT-4-era set	TF-IDF word 1–2gram	0.969	stripped	—
all pairs, comments kept	TF-IDF word 1–2gram	0.962	kept	inflated by the comment channel
working, comments kept	TF-IDF word 1–2gram	0.979	kept	inflated by the comment channel
inert, comments kept	TF-IDF word 1–2gram	0.955	kept	inflated by the comment channel
transfer, comments kept	TF-IDF word 1–2gram	0.994	kept	inflated by the comment channel
same, raw artifact	code length only	0.826	kept	—

a literal record of authorship. Stripped, the matched-protocol TF-IDF AUROC is 0.8560, beside $B = 0.871$ and the Koran-split 0.806, all three on stripped code. The other cross-generator transfer rows, stripped, are 0.866 (o3-mini to claude-3-7-sonnet, 82 test pairs), 0.844 (the reverse) and 0.764 (o1-mini to gemini-2.0-flash).

A.4 Pair similarity, and why the same-generator control needs two arms

An attack class built by editing its own honest solution changes pair similarity as well as authorship (Table 4); the independent arm sits in the stored benchmark's regime and if anything slightly further from it, so it cannot manufacture a low S by making the classes alike.

Table 4: Character similarity of honest/attack pairs, comments stripped. The edit arm's classes are near-copies; the independent arm reproduces the stored benchmark's regime, where the two classes are largely independent programs.

Attack class	mean sim.	> 0.95	< 0.60	changes vs. the benchmark
stored benchmark	0.606	3.4%	45.4%	— (the regime to match)
edit (shown its own solution)	0.928	64.2%	4.0%	authorship *and* similarity
independent (problem only)	0.659	3.4%	34.8%	**authorship only**

A.5 Robustness of the same-generator rows

Intervals in the Gate S paragraph of §3 are bootstraps over problems, resampling S and B' jointly for ρ; this replaced a ± 2 fold-sd rule before the retry pool was read. Every ρ point we quote is the ratio

of the fold-mean AUROCs and every joint interval beside it is the bootstrap of the pooled out-of-fold ρ, which is the same quantity computed two ways: the pooled points are 0.86 (strict, the row whose interval Table 1 quotes), 0.390 (prompt-matched headline), 0.662 (prompt-matched strict) and 0.097 (the second generator). Five fold seeds move the strict row's ρ over 0.85–0.92. Retention is not the artifact: survivors and non-survivors separate at only 0.62, and S with non-firing attempts on the same problems is 0.7595 against 0.7687. The edit arm bounds S under near-copies (similarity 0.928, Table 4): 0.6266 over 427 pairs, $\rho = 0.371$. In arm 1, where only the honest half was regenerated, $G = 0.962$ against $B' = 0.874$ on that arm's $n = 668$ eligible problems, and TF-IDF gains +0.088 while the length carrier collapses to 0.489. Our own attack prompt halves our single-shot firing rate: 52.0% of our test-passing draws offer a public test input as the trigger, which cannot diverge from the honest solution, and those fire 0.15% against 5.13% for novel triggers; no stored working backdoor has such a trigger (0 of 3,188). The rest is retry-and-keep.

A.6 The prompt-matched arm under the masks, and the second generator

Under prompt matching the identifier mask — the only mask of §3 that moved S — takes the headline row to 0.6264 ($\rho = 0.342$) where it takes the original honest class on the same 586 problems to 0.7046: most of what identifier normalisation removed is what prompt matching removes. That masked row is the one that decides whether the residue is structure or naming, and under prompt matching it is band-ambiguous: its joint interval, 0.280–0.394, puts 40% of its mass in the collapse band. A character model (3–5-grams, same classifier and folds) retains more than the word tokeniser, 0.6817 ($\rho = 0.493$), so the drop is not an artefact of tokenisation. On the strict pairs the matched row is $S = 0.7113$ ($n = 96$, $\rho = 0.697$, 0.494–0.856). The second generator's pairs are more self-similar than the first's (0.724 mean against 0.659), so S was recomputed inside similarity strata: it stays 0.4737–0.5142 in every stratum, including the 0.451-similarity stratum, which is further from its honest class than the stored benchmark's own pairs, while B' holds 0.7884–0.8334. The collapse is not bought with near-copies. Its prompt-matched rows move S to 0.5399 ($\rho = 0.113$, $n = 562$) and put the prompt-only floor at 0.4918, an interval containing chance. The mechanism is visible in the programs: the layout vocabulary the first generator's residue is made of — `def main`, `sys.stdin`, `data[` — is present in 0.000 of the second generator's programs in *all three* of its classes, which read standard input with `input(` in 1.000 of programs on every side, so there is no layout difference left for the classifier to read.

A.7 The monitor rows, and what bounds the second monitor

The monitor that wrote neither class scores 0.5577 on the same-generator headline pairs and 0.8824 on the stored pairs over the same problems, 0.7871 and 0.8832 on the strict pairs. Its headline rows are a bound rather than a point: a 512-token reply ceiling left 628 of 1,246 same-generator replies unparseable, and on the stored side the missing share differs by class (50.1% honest against 23.4% attack), a bias of unknown sign. Re-running the strict pairs with a 1,024-token ceiling prices it: missingness falls to 2 of 198 and monitor-ρ moves from 0.749 to 0.766, inside the bootstrap interval. On the stored pairs both monitors beat the bag-of-words baseline over the same problems ($+0.09$ and $+0.15$ for the first, $+0.02$ and $+0.09$ for the second): Figure 1's ranking is a statement about those monitors on that split, not a claim that a bag of words beats a monitor here. Classifier and monitor are independent pair by pair on the same-generator pairs ($\phi = 0.04$): they are not two readings of one signal.

A.8 The two generators' funnel

Both generators were run through the identical pipeline. Our honest draws pass the public tests at 0.7701 and 0.7278 of 1,444 problems; the second generator's attack draws pass at 689 of 1,444 and 35 of those fire, below the 50-pair floor `gate_s_samegen.py` requires, which is why it has no strict row and the two generators are compared on the `passes` eligibility only. The two compared rows are not equally inert either: 2 of the 611 attacks in the second generator's row fire (99.67% inert) against 14 of 623 (97.75%), the other 33 of its firing draws sitting on problems where its own honest program failed the tests and which Gate S therefore drops. The single-shot firing rate is a property of the protocol rather than of either model: 52.0% of test-passing draws offer a public test input as their trigger.

A.9 The five same-generator agentic settings

Figure 2 shows each setting's content-blind floor against its own permutation null.

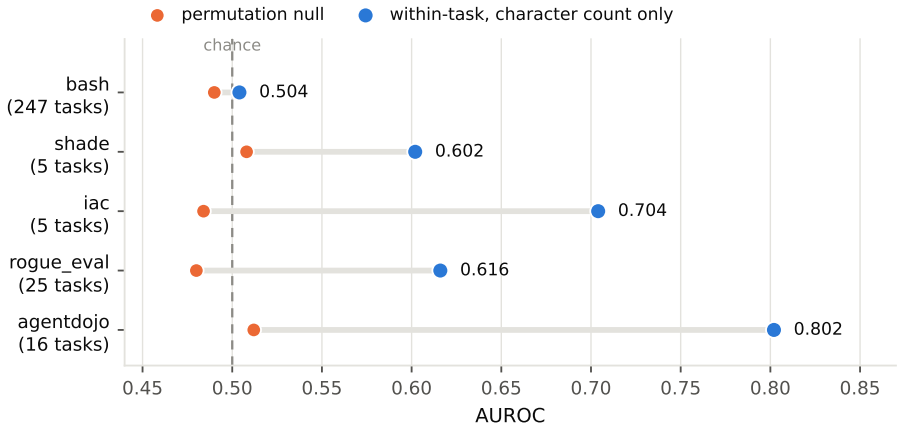

Figure 2: **The structural floor in five same-generator agentic settings.** Each setting's content-blind classifier — six count features, no content read — scored *within task* against its own permutation null. No authorship confound is possible here: claude-sonnet-4, gemini-2.5-pro and gpt-5 write both classes in all five. `bash` sits on its null at 0.504; `agentdojo` reaches 0.802 from character count alone. **The task-group count beside each setting matters:** `shade` and `iac` average over five task groups each while `bash` averages over 247, so these floors are not equally well determined. We report the within-task column because our own pooled numbers partly broke on task composition, and we broke them ourselves. The permutation nulls shown were computed with a pooled estimator rather than the folded within-task statistic; they bound the chance level but are not a like-for-like null for each dot. These corpora declare no licence; we redistribute nothing and report aggregates only.

A.10 The release

The supplementary bundle holds the same-generator split (1,444 honest programs, 1,112 edit-arm and 1,112 independent-arm attack programs written by one model, plus every one of the 11,565 draws of the independent arm's retry pool), both eligibility rules as pair lists under an outcome-blind selection (623 passing and 14 firing independent-arm pairs) and under the benchmark's own survivor selection (1,046 and 99, or 106 counting a crash on the trigger as firing), the 24 scripts behind the measurements — generation for both generators, the test harness, the trigger verifier, the selection rules, the prompt-matched arm, the monitor rows and each remaining measurement script — a dataset card, and a manifest of row counts and checksums. The licence, MIT, is a file inside the bundle and covers the data and the scripts alike.

A.11 The test harness and the pass flag

Gate S requires executing generated code, so we built a runner that feeds each shipped input case to a solution under per-case timeouts and address-space limits. It agrees with the artifact's own `solution_passes_tests` on 95.84% of the analysis pool and is strictly conservative there — it never credits a solution the artifact rejects (0 such cases). We use *our own* flag for both classes: a checker stricter on one class than the other would manufacture exactly the artifact this paper documents. We do not length-match either class; the untrusted model chooses its own trajectory length. The headline does not depend on that choice — under the artifact's flag $B = 0.871$ at $n = 1{,}444$, under ours $B = 0.873$ at $n = 1{,}406$, within one standard deviation of each other. The monitor scores in Figure 1 are `runs/x0/monitor-summary.json` of [9]'s MIT release, used under its notice. Two interpreter-era artifacts in the upstream solutions fall on opposite classes: a `stdin` without `.buffer`, fixed, and `fractions.gcd`, removed in Python 3.9, not fixable. B is exposed to them; the same-generator arm, whose code we generated and ran ourselves, is immune.

A.12 The token test, run and failed

We attempted to confirm the token-level mechanism of [14] in our setting and it does not replicate: separation -0.013 — the wrong sign — at permutation $p = 0.956$. The reason is register. Their human class is general-purpose repository code, whereas *both* of our classes are competitive-programming code, so an identifier axis that separates general human code from model code does not separate these two populations. Only 8 of their 10 canonical tokens are in our vocabulary, so the test is underpowered; but the point estimate has the wrong sign, which is not what an underpowered true effect looks like. We report it because a reader who knows that work may run exactly this test.

A.13 A third channel, noted rather than developed

In the same five agentic settings a fifteen-word regular expression over the untrusted policy's own scratchpad reaches 0.76–0.82, because prompted attack policies restate their side task in the reasoning the monitor is given. We note this only to bound the claim: the instrument belongs to [13], which validates it properly against an LLM judge and a length-matched null, and we defer to that treatment.